

Selenium

Mature WebDriver ecosystem for browser automation with Java, Python, C#, JavaScript and other languages.

What it is and when to use it

Selenium automates browsers through the WebDriver standard and remains strong in enterprise and multi-language ecosystems.

Official site and documentation

Use the official Selenium source as the primary reference: <https://www.selenium.dev/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Prerequisites

Choose the project language and install its runtime/JDK/SDK. For Java configure JDK plus Maven/Gradle; for Python use Python and a virtual environment.

Installation and drivers

Add Selenium through the language package manager. Selenium Manager can resolve drivers in many modern setups; verify browser/driver compatibility when startup fails.

First project

Start WebDriver, navigate, locate elements, perform actions, assert results and close the driver safely.

Locators and waits

Prefer stable IDs/test attributes and explicit waits. Avoid fixed sleeps as synchronization strategy.

Page Object and architecture

Separate page behavior, data, configuration and assertions. Use Page Objects to model behavior, not only selectors.

Grid and parallel execution

Selenium Grid distributes sessions across browsers and machines; size parallelism to data isolation and environment capacity.

CI/CD and evidence

Run headless when appropriate and publish reports, screenshots and browser/driver logs.

Troubleshooting

Common issues include driver mismatch, stale elements, synchronization, frames, windows and shadow DOM.

How to start safely

Start from the official Selenium site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Selenium, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Selenium fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Selenium still delivers value as the product evolves.

Common issues and troubleshooting

When Selenium fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.